

Constructor

Constructor

يعني اي constructor واي فايدته اصلا

بص الموضوع بدا من ونا كنت بعمل class و الكلاس دي فيها مجموعة fields دلوقتي انا عايز ابعت قيم ال fields دي عادي سهله هعمل فانكشن تاخد برامترز بعدد ال fields دي واعملها كال وابتعلها القيم وخلص ماشي طب لا انا عايز اول م اعمل ال object ابعت القيم مره واحده اه يعني انا عايز اعمل initialization في نفس سطر اللي ال definition ال object بتاعي بس هو دا كل اهميه ال constructor اني اعمل initialization لل fields ونا بكريت ال object بتاعي من غير بق م اعمل فانكشن وتاخد برامترز واعملها كال لا بعمل ال constructor بتاعي

اذن ال Constructor هو كود يتم تنفيذه تلقائيًا عند إنشاء ال Object، وغالبًا يُستخدم لعمل Initialization لل Fields وتجهيز ال Object ليكون في حالة صالحة للاستخدام.

ف بدل الشكل دا

```
static void Main(string[] args)
{

    var d1 = new Date();
    d1.setdate(1, 1, 1);
    Console.WriteLine(d1.DisplayDate());

}

public class Date
{
    public int Day;
    public int Month;
    public int Year;
    public void setdate(int day,int month,int year)
    {
        Day = day;
        Month = month;
        Year = year;
    }
    public string DisplayDate()
    {
        return $"{Day.ToString().PadLeft(2, '0')} {Month.ToString().PadLeft(2, '0')}
{Year.ToString().PadLeft(4, '0')}";
    }
}
```

بيبق كد

```
static void Main(string[] args)
{

    var d1 = new Date(1, 1, 1); //intialization in definition of object
    Console.WriteLine(d1.DisplayDate());

}

public class Date
{
    public int Day;
    public int Month;
    public int Year;
    public Date(int day,int month,int year)
    {
        Day = day;
```

```
        Month = month;
        Year = year;
    }
    public string DisplayDate()
    {
        return $"{Day.ToString().PadLeft(2, '0')} {Month.ToString().PadLeft(2, '0')}
{Year.ToString().PadLeft(4, '0')}";
    }
}
```

اذن نستنتج ان ال constructor بيتكتب كد

```
<AccessModifer>  ClassName (# of parameters)
{
    # series of statements;
}
```

بدون اي return datatype ولا حتى void

انواع ال constructors

1. Default Constructor

```
class Person
{
    public Person()
    {
        Console.WriteLine("Default Constructor");
    }
}
```

الاستخدام:

```
Person p = new Person();
```

ممکن معمولهوش في العادي لانه يبيق موجود بشكل default بس لازم اعمله لو عندي اي Parameterized Constructors ثانيه عشان ميحصلش مشكله ونا بكرت object زي العادي يعني زي كد

```
Person p = new Person();
```

2. Parameterized Constructor

يستقبل معاملات.

```
class Person
{
    string name;

    public Person(string n)
    {
        name = n;
    }
}
```

الاستخدام:

```
Person p = new Person("Menna");
```

طبعا دا اللي بستعمله عشان اعمل initialization لل fields اللي عندي

3. Copy Constructor “مش موجود صريح في اللغة”

```
class Person
{
    public string Name;

    public Person(Person p)
    {
        Name = p.Name;
    }
}
```

الاستخدام:

```
Person p1 = new Person();
Person p2 = new Person(p1);
```

4. Static Constructor

```
class Person
{
    static Person()
    {
        Console.WriteLine("Static Constructor");
    }
}
```

خصائصه:

- لا يأخذ Parameters.
- لا يكتب معه Access Modifier.
- ينفذ مرة واحدة فقط طوال عمر البرنامج.
- تهيئة الأعضاء الـ Static مرة واحدة فقط
- يُنفذ مرة واحدة فقط قبل أول استخدام للكلاس سواء عند إنشاء أول Object أو عند الوصول لأول Static Member.
- أول مره بعمل object ييتمنفذ بعد كد في اي object ثاني مش هيتنفذ

مثال:

```
class Person
{
    static Person()
    {
        Console.WriteLine("Static Constructor");
    }

    public Person()
    {
        Console.WriteLine("Normal Constructor");
    }
}
```

```
Person obj1=new Person();
Person obj2=new Person();
```

الـ Output:

Static Constructor
Normal Constructor
Normal Constructor

ال Static Constructor تنفذ مرة واحدة فقط عند إنشاء `obj1`.

أما عند إنشاء `obj2`:

- ال Static Constructor لن يتنفذ مرة أخرى.
- ال Normal Constructor فقط هو اللي هيتنفذ.

5. Private Constructor

```
class Database
{
    private Database()
    {
    }
}
```

يستخدم غالبًا مع Design Patterns مثل Singleton.
منع إنشاء كائنات من خارج الكلاس

Constructor Chaining

```
class Person
{
    public Person(string name)
    {
        Console.WriteLine(name);
    }

    public Person() : this("Unknown")
    {
    }
}
```

Constructor Chaining – Definition

Constructor Chaining is the process of calling one constructor from another constructor in order to reuse initialization logic.

It allows a constructor to invoke:

- Another constructor in the **same class** using `this`, or
- A constructor in the **base (parent) class** using `base`.

Purpose:

- Avoid code duplication.
- Ensure proper object initialization.
- Maintain clean and organized constructor logic.

In simple words:

Constructor Chaining means **linking constructors together**, so when one constructor runs, it automatically calls another constructor first.

Types in C#:

1 Using `this` → **Same class constructor**

2 Using `base` → Parent class constructor

النوع	الشكل	الاستخدام
Default Constructor	<code>public Person(){} </code>	إنشاء كائن بدون تمرير قيم
Parameterized Constructor	<code>public Person(string name){} </code>	تهيئة الكائن بقيم عند الإنشاء
Copy Constructor	<code>public Person(Person p){} </code>	إنشاء كائن جديد بنسخ بيانات كائن آخر
Static Constructor	<code>static Person(){} </code>	مرة واحدة فقط Static تهيئة الأعضاء الـ
Private Constructor	<code>private Person(){} </code>	منع إنشاء كائنات من خارج الكلاس
Constructor Chaining	<code>: this(...)</code>	آخر من نفس الكلاس Constructor استدعاء

Constructor Overloading

```
class Person
{
    public Person()
    {
    }

    public Person(string name)
    {
    }

    public Person(string name, int age)
    {
    }
}
```

وجود أكثر من Constructor داخل نفس الكلاس بشرط اختلاف قائمة الـ Parameters.
نفس فكرة الـ Method overloading كد الـ compiler ييقرر يحدد اي واحد ع حسب عدد او ترتيب او نوع الـ parameters اللي انا باعتها